

Sprawozdanie z projektu nr 1

Asymetryczny Problem Komiwożera (ATSP)

Projektowanie Efektywnych algorytmów

Karol Gąsior / 280913

23 kwietnia 2026

Spis treści

1	Wstęp teoretyczny	3
1.1	Algorytm podziału i ograniczeń (Branch and Bound)	3
2	Przykład praktyczny algorytmu B&B (Depth-First Search) dla $N = 3$	3
3	Opis implementacji algorytmu i zastosowanych struktur	5
3.1	Kluczowe struktury danych	6
3.2	Funkcja obliczająca ograniczenia dla metody B&B (Algorytm Little'a) . . .	6

1 Wstęp teoretyczny

1.1 Algorytm podziału i ograniczeń (Branch and Bound)

W ramach drugiego etapu projektu zaimplementowano algorytm podziału i ograniczeń (ang. *Branch and Bound*, B&B) w celu rozwiązania asymetrycznego problemu komiwojażera (ATSP)[cite: 2, 3]. Metoda ta jest dokładnym algorytmem optymalizacyjnym, który w przeciwieństwie do przeglądu zupełnego nie sprawdza każdej możliwej permutacji w sposób naiwny.

Działanie algorytmu opiera się na budowaniu drzewa przestrzeni rozwiązań. W każdym węźle drzewa obliczane jest tzw. dolne ograniczenie (ang. *Lower Bound*, LB) kosztu dla wszystkich ścieżek wychodzących z danego węzła. Jeżeli wartość LB danej gałęzi przekracza lub jest równa kosztowi najlepszego dotychczas znajdującego rozwiązania (górne ograniczenie), cała gałąź jest odrzucana (odcinana). Zgodnie z założeniami projektowymi, algorytm został wyposażony w trzy metody przeglądania tak zbudowanej przestrzeni: *breadth-first-search* (przeszukiwanie wszerz), *depth-first-search* (przeszukiwanie w głąb) oraz *lowest-cost* (najlepszy najpierw)[cite: 6].

2 Przykład praktyczny algorytmu B&B (Depth-First Search) dla $N = 3$

Poniżej przedstawiono działanie algorytmu krok po kroku dla przykładowej instancji problemu o małej wartości N [cite: 14]. Przyjęto strategię przeszukiwania w głąb (DFS), a dolne ograniczenie wyznaczane jest metodą Little’a. Niech początkowa asymetryczna macierz kosztów M wynosi:

$$M = \begin{bmatrix} \infty & 10 & 15 \\ 5 & \infty & 9 \\ 6 & 12 & \infty \end{bmatrix}$$

Krok 1: Węzeł początkowy (Root) i wyznaczenie LB

- **Redukcja wierszy:** W każdym wierszu znajdujemy wartość minimalną i odejmujemy ją od pozostałych elementów wiersza. Minimum wiersza 0: 10. Po odjęciu: $[\infty, 0, 5]$. Minimum wiersza 1: 5. Po odjęciu: $[0, \infty, 4]$. Minimum wiersza 2: 6. Po

odjęciu: $[0, 6, \infty]$. Suma redukcji wierszy: $10 + 5 + 6 = 21$.

- **Redukcja kolumn:** Analogicznie odejmujemy minima z kolumn (dla zredukowanej już macierzy). Minimum kolumny 0: 0. Minimum kolumny 1: 0. Minimum kolumny 2: 4. Po odjęciu: kolumna 2 to $[1, 0, \infty]$. Suma redukcji kolumn: $0 + 0 + 4 = 4$.
- **Macierz zredukowana węzła początkowego M' :**

$$M' = \begin{bmatrix} \infty & 0 & 1 \\ 0 & \infty & 0 \\ 0 & 6 & \infty \end{bmatrix}$$

- **Koszt węzła (LB):** $21 + 4 = 25$. Węzeł trafia na Stack.

Krok 2: Rozgałęzienie (Branching) z wierzchołka 0 Pobieramy węzeł ze stosu. Możemy udać się do miasta 1 lub 2. Zgodnie z metodą DFS, badamy obydwie opcje i umieszczamy je na stosie.

Węzeł Dziecko A (ścieżka $0 \rightarrow 1$):

- Kopiujemy macierz M' . Blokujemy wiersz 0 i kolumnę 1 (ustawiamy na ∞). Blokujemy powrót (ustawiamy $M'[1][0] = \infty$).
- Macierz przed ponowną redukcją:

$$M_A = \begin{bmatrix} \infty & \infty & \infty \\ \infty & \infty & 0 \\ 0 & \infty & \infty \end{bmatrix}$$

- Macierz jest już w pełni zredukowana (w każdym niezablokowanym wierszu i kolumnie jest zero). Koszt redukcji = 0.
- Nowe $LB_A = LB_{poprzednie} (25) + \text{Koszt przejścia } M'[0][1] (0) + \text{Koszt redukcji } (0) = 25$.

Węzeł Dziecko B (ścieżka $0 \rightarrow 2$):

- Blokujemy wiersz 0, kolumnę 2 oraz powrót $M'[2][0]$.

- Macierz przed redukcją:

$$M_B = \begin{bmatrix} \infty & \infty & \infty \\ 0 & \infty & \infty \\ \infty & 6 & \infty \end{bmatrix}$$

- Redukcja wiersza 2 (minimum to 6). Po odjęciu wiersz 2 to $[\infty, 0, \infty]$. Suma redukcji = 6.
- Nowe $LB_B = LB_{poprzednie}$ (25) + Koszt przejścia $M'[0][2]$ (1) + Koszt redukcji (6) = 32.

Zarządzanie stosem: Na stos (LIFO) odkładamy Węzeł B, a następnie Węzeł A.

Krok 3: Eksploracja DFS (Węzeł A)

- Ściągamy ze stosu Węzeł A (ścieżka: $0 \rightarrow 1$, $LB = 25$). Ponieważ nie odwiedziono jeszcze wszystkich miast, generujemy jedyne możliwe dziecko: ścieżkę $0 \rightarrow 1 \rightarrow 2$.
- Macierz zostaje całkowicie wyzerowana/zablokowana, koszt nie rośnie. Kompletna trasa to $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$.
- Ustawiamy najlepszy dotychczasowy koszt rozwiązania (*upper bound*) na 25.

Krok 4: Próba eksploracji Węzła B i odcięcie (Pruning)

- Ściągamy ze stosu Węzeł B (ścieżka $0 \rightarrow 2$, $LB = 32$).
- Ponieważ LB_B (32) jest większe niż dotychczas znalezione najlepsze rozwiązanie (25), algorytm natychmiast odrzuca tę gałąź bez dalszej eksploracji. Stos jest pusty, algorytm kończy działanie.

3 Opis implementacji algorytmu i zastosowanych struktur

Implementacji dokonano w języku C++ z zachowaniem obiektowego paradygmatu programowania[cite: 10]. Mając na uwadze optymalizację pamięciową, wszystkie używane struktury danych są alokowane dynamicznie i zależą wyłącznie od aktualnie wprowadzonego rozmiaru problemu[cite: 5].

3.1 Kluczowe struktury danych

- **Struktura Node:** Fundamentalna klasa reprezentująca węzeł w drzewie przestrzeni rozwiązań. Przechowuje zredukowaną macierz kosztów przejść (reprezentującą stan problemu w danym punkcie), listę odwiedzonych miast (wektor ścieżki), indeks obecnego miasta, głębokość w drzewie oraz skalkulowaną dla tego węzła wartość dolnego ograniczenia (*Lower Bound*). Przechowywanie instancji macierzy bezpośrednio w węźle ułatwia implementację metod DFS/BFS i izoluje stany między rozgałęzieniami.
- **Struktura Stack:** Dynamiczna struktura danych typu LIFO (*Last-In, First-Out*). Jest kluczowym elementem implementacji przeglądania przestrzeni metodą *depth-first-search*[cite: 6]. Zastosowanie stosu pozwala na sztuczne wywołania rekurencyjne (iteracyjne zagłębianie się w drzewo) bez ryzyka przepełnienia stosu systemowego (*Stack Overflow*) dla instancji o większym wymiarze N . W przypadku strategii *breadth-first-search* zamiast stosu wykorzystywana jest struktura kolejki (*Queue*), a dla *lowest-cost* - kolejka priorytetowa (*Priority Queue*).

3.2 Funkcja obliczająca ograniczenia dla metody B&B (Algorytm Little'a)

Funkcja wyznaczająca dolne ograniczenie w każdym węźle opiera się na algorytmie redukcji macierzy (metoda Little'a)[cite: 15]. Procedura ta składa się z następujących etapów:

1. **Zabezpieczenie przed cyklami podrzędnymi:** Przed rozpoczęciem redukcji dla tworzonego węzła-dziecka, ścieżka z miasta i (rodzic) do j (dziecko) jest usuwana z dostępnych opcji. Cały wiersz i oraz kolumna j w macierzy są wypełniane wartościami ∞ . Dodatkowo, aby zapobiec przedwczesnemu zamknięciu cyklu, krawędź powrotna z j do i również otrzymuje koszt ∞ .
2. **Redukcja wierszowa:** Algorytm iteruje po wszystkich niezablokowanych wierszach. Dla każdego znajduje element o najmniejszej wartości i odejmuje go od wszystkich elementów w tym wierszu. Zliczana jest suma odjętych minimów.
3. **Redukcja kolumnowa:** Proces jest powtarzany dla kolumn nowej macierzy. Minima z niezablokowanych kolumn są odejmowane, a ich wartości powiększają sumę całkowitą.

4. **Ewaluacja kosztu:** Ostateczna wartość ograniczenia dolnego (*Lower Bound*) dla nowego węzła obliczana jest jako suma trzech elementów: LB węzła rodzica, kosztu krawędzi (i, j) z pierwotnej macierzy rodzica oraz całkowitego kosztu obecnej redukcji wierszowo-kolumnowej.